Khazar University

Student: Museib Aliyev
Course: Computer Engineering
Subject: Image Processing
Github link: https://github.com/Museib/ImageProcessingFinalProject

Final Project: Developing and Tuning a Deep Learning Pipeline for Real-World Image Classification

---

Abstract

Deep learning has revolutionized the field of computer vision, enabling highly accurate image classification across a wide range of applications. In this project, we explore and document the end-to-end development of a convolutional neural network (CNN) for image classification using the CIFAR-10 dataset as a prototypical example. The work involves data preprocessing, data augmentation, construction of a CNN architecture, hyperparameter tuning, and the application of regularization to avoid overfitting. We then evaluate the model using metrics such as accuracy, confusion matrix, and classification report, and visualize training curves to better understand the training and validation processes.

By integrating best practices—such as dropout, L2 regularization, early stopping, and data augmentation—we were able to build a robust image classification pipeline that can generalize well to unseen data. We also briefly discuss the utility of transfer learning and how it can be used to improve performance or reduce training time on smaller datasets. This report covers the complete workflow, from dataset loading to performance evaluation, along with an in-depth discussion of the results, challenges faced, and limitations. Our findings underscore the importance of carefully curating hyperparameters and employing regularization techniques to achieve a good balance between underfitting and overfitting.

---

1. Introduction

The rapid growth of deep learning techniques has drastically changed the landscape of computer vision, enabling remarkable performance improvements in image classification tasks. Convolutional Neural Networks (CNNs) in particular have become the de facto standard for tackling image-related problems due to their ability to learn hierarchical, spatially invariant features. Since the seminal work of AlexNet in 2012, CNN-based architectures—ranging from VGG and ResNet to MobileNet—have demonstrated consistently strong results on benchmarks such as CIFAR-10, CIFAR-100, and ImageNet.

Despite the advancements, building a deep learning model that performs well in practice requires careful attention to every step of the workflow. A thorough approach involves selecting a suitable dataset, performing preprocessing, applying data augmentation strategies, and choosing the right architecture or pre-trained backbone when relevant. Further refinements such as hyperparameter tuning, dropout, L2 regularization, and early stopping help mitigate overfitting and improve generalizability.

In this project, we focus on designing and tuning a CNN pipeline using the CIFAR-10 dataset, which contains 60,000 images from ten different classes (airplane, automobile, bird, cat, deer, dog, frog, horse, ship, truck). The dataset is small enough to allow for rapid experimentation, yet sufficiently challenging to illustrate the common pitfalls and best

practices in deep learning pipelines. Additionally, we illustrate how optional transfer learning with a pre-trained model (e.g., MobileNetV2) might be beneficial, especially when data is limited or training time is constrained.

Our ultimate goal is to produce a well-documented, reproducible pipeline that can be extended to other datasets or tasks, covering everything from data preprocessing to model evaluation. The results presented here highlight the importance of systematic experimentation and the impact of various hyperparameters. By systematically explaining the methodology, results, challenges, limitations, and possible improvements, we hope to provide a solid framework for future image classification endeavors.

---

2. Objective

The primary objective of this project is to **develop and tune a deep learning pipeline for real-world image classification**. To break this further down, we aim to:

1. **Build a Convolutional Neural Network (CNN) from scratch** for image classification tasks, while also demonstrating an alternative approach using **transfer learning**.
2. **Explore hyperparameter tuning**—through grid search and random search—to optimize parameters such as learning rate, dropout rate, and L2 regularization strength.
3. **Utilize data augmentation** techniques to enrich the training data and improve model generalization.
4. **Integrate regularization strategies**—including dropout, L2 weight decay, and early stopping—to minimize overfitting and maximize test performance.
5. **Perform in-depth model evaluation**, reporting metrics such as accuracy, precision, recall, F1-score, and confusion matrices to glean detailed insights into performance.

By satisfying these objectives, we provide a robust and adaptable blueprint for training deep learning models on image classification tasks, backed by strong theoretical underpinnings and practical implementation details.

---

3. Methodology

3.1 Dataset Preparation and Preprocessing

Dataset Selection: We chose the CIFAR-10 dataset for its balance between complexity and manageability. CIFAR-10 includes 32×32 color images across ten object classes. It is a widely recognized benchmark that allows for quick iteration.

Data Loading: The dataset is readily accessible via TensorFlow's Keras API. We loaded the training and test sets, amounting to 50,000 training images and 10,000 test images.

Normalization: Each pixel's intensity values were normalized to the 0,10, 10,1 range by dividing by 255.0. Normalization is critical to ensure stable gradients and faster convergence during training.

Label Encoding: Since CIFAR-10 provides integer labels (0 through 9), we converted these labels into one-hot encodings using Keras' to_categorical function, thus creating a 10-element binary vector for each image label.

Training-Validation Split: To monitor generalization during training, we extracted 20% of the training data as a validation set using train_test_split from the scikit-learn library. This left us with 40,000 training images and 10,000 validation images.

3.2 Data Augmentation

Data augmentation helps CNNs generalize by artificially increasing dataset variety. We employed **ImageDataGenerator** from Keras, applying a rotation range of 15 degrees, horizontal flips, and random width and height shifts of up to 10%. These augmentations mimic natural variations in real-world scenarios, preventing the model from memorizing specific training samples.

3.3 Model Development

3.3.1 Baseline CNN

The core of our pipeline is a custom CNN that progressively learns spatial hierarchies:

1. **Convolution Layers:** We started with two consecutive convolutional layers of 32 filters each, followed by a max pooling layer and dropout.
2. **Depth Expansion:** We similarly added two sets of convolutional layers (64 and 128 filters, respectively), each followed by pooling and dropout to reduce feature map dimensionality and mitigate overfitting.
3. **Fully Connected Layers:** We then flattened the feature maps and added a 256-unit dense layer (with ReLU activation), followed by another dropout layer. The final layer has 10 units with a softmax activation.

Each convolutional layer includes kernel L2 regularization, specified via kernel_regularizer=regularizers.l2(l2_reg) to further combat overfitting.

3.3.2 Transfer Learning (Optional)

As an alternative approach, we also outlined the steps for employing **MobileNetV2** trained on ImageNet. We froze its convolutional layers to retain learned features, replaced its final layers with a global average pooling and dense classification layer, and fine-tuned only the top layers. This approach can be particularly advantageous if one works with smaller datasets or if rapid convergence is desired.

3.4 Regularization Techniques

Dropout: Randomly zeroing out a proportion of neuron outputs helps prevent complex co-adaptations. By default, we used a dropout rate of 0.4, although this rate was also a subject of hyperparameter tuning.

L2 Regularization (Weight Decay): Adding L2 penalty constraints the magnitude of weights, preventing them from growing excessively. We used L2 penalties (1e-4, 1e-3, or 1e-2 in the hyperparameter search) for convolutional and dense layers.

Early Stopping: A callback that monitors the validation loss and ceases training once performance fails to improve for a set number of epochs (patience). This prevents overfitting by avoiding unnecessary training.

3.5 Hyperparameter Tuning

To systematically explore hyperparameters, we utilized **Keras Tuner**. Two main strategies were tested:

1. **Grid Search:** We systematically combined small sets of hyperparameters (e.g., learning rate, dropout rate, L2 penalty). This can be computationally expensive if the search space grows large.
2. **Random Search:** We used kt.RandomSearch to sample from discrete sets of values for dropout rate, L2 penalty, and learning rate. Although random search lacks the exhaustive coverage of grid search, it often finds competitive or superior solutions with fewer trials.

For each trial, we trained the model for up to 5 epochs and used an early-stopping callback. The best hyperparameters were recorded based on highest validation accuracy.

3.6 Model Training and Evaluation

With the chosen hyperparameters, the final model was compiled using the **Adam** optimizer. We used categorical_crossentropy as the loss function and tracked both training and validation accuracy. After training, we evaluated the final model on the held-out test set.

We further analyzed the model with:

- **Accuracy**: The proportion of test samples correctly classified.
- **Confusion Matrix**: Provides a class-by-class breakdown to highlight common misclassifications.
- **Classification Report**: Summarizes precision, recall, and F1-score, offering a more detailed analysis of performance.

Finally, training curves were generated to visualize how the model's accuracy and loss evolved over epochs.

---

4. Results, Visualizations, and Key Findings

4.1 Training and Validation Curves

One of the first indicators of whether a model is learning effectively is its training and validation loss/accuracy curves. Over 20 epochs, our CNN displayed the following characteristics:

- **Accuracy**: Training accuracy steadily increased from roughly 30% at the start to over 85% by the 20th epoch. Validation accuracy trailed but followed a similar upward trajectory, peaking around 80% to 82%.
- **Loss**: Training loss decreased consistently, while validation loss initially fell and then plateaued. The early stopping callback prevented further overfitting after detecting stagnation in validation loss.

4.2 Test Accuracy

At the end of training, the test accuracy reached **~80–82%**, depending on the chosen hyperparameters. This is a respectable figure for a relatively simple CNN architecture trained from scratch on CIFAR-10 without extensive optimization or large-scale data augmentation.

4.3 Confusion Matrix

The confusion matrix displayed a few interesting insights:

- Classes like “automobile” and “truck” sometimes get confused, which is common due to their shared semantic features.
- Animal classes such as “cat,” “dog,” and “frog” also resulted in some cross-class confusion.
- Relatively distinct classes (e.g., “airplane” vs. “ship”) achieved very high classification accuracy.

These results suggest that the model is learning robust representations but may still struggle with visually similar classes.

4.4 Classification Report

A more detailed look at precision, recall, and F1-score for each class indicates that some classes surpass 85% F1-score, while others hover around 70%. The average weighted F1-score aligns closely with the overall accuracy, showcasing that our model is consistent in balancing precision and recall across classes.

4.5 Key Findings

1. **Data Augmentation** significantly improved our model’s generalization, particularly preventing overfitting that would otherwise occur with a relatively small dataset.
2. **Regularization** strategies such as dropout and L2 weight decay proved essential in maintaining a stable validation accuracy and in preventing the model from memorizing training samples.
3. **Hyperparameter Tuning** with random search identified a favorable combination of learning rate (1e-3), dropout rate (0.4), and L2 penalty (1e-4). Slight variations in these hyperparameters often led to visible changes in final accuracy, highlighting their importance.
4. **Transfer Learning** (when tested on a smaller subset of data) provided a faster convergence and higher performance early on, although the final numbers can be comparable if the baseline CNN is well-tuned.

---

5. Discussion of Challenges Faced and Solutions Implemented

5.1 Challenge: Overfitting and Limited Data

A common issue in training CNNs on smaller datasets (like CIFAR-10 with only 50,000 training images) is the risk of overfitting, especially as networks grow deeper. We addressed overfitting through multiple routes:

- **Dropout**: By randomly removing a percentage of neurons during training, we forced the network to develop more robust features.
- **Data Augmentation**: We artificially increased the training set’s diversity by applying random transformations. This effectively gave the model more unique samples to learn from.
- **Early Stopping**: By monitoring validation loss, we halted training before the model could memorize the dataset.

These measures collectively helped our network generalize better without stagnating in local minima or memorizing training examples.

5.2 Challenge: Hyperparameter Space Exploration

Manually searching for hyperparameters can be overwhelming due to the vast combinations of learning rates, batch sizes, layer sizes, and regularization coefficients. To handle this complexity, we employed:

- **Random Search via Keras Tuner**: Random search is straightforward to set up and can discover surprisingly optimal settings more quickly than naive methods.
- **Small Batches of Trials**: Given computational constraints, we limited the search to a modest number of trials (e.g., five), thereby balancing exploration with practical runtime.

We discovered that a moderate learning rate of 1e-3, a dropout rate of 0.4, and a weight decay of 1e-4 served as a good fit for our baseline CNN architecture.

5.3 Challenge: Computational Resources

Deep learning model training can be time-intensive, especially when employing multiple hyperparameter trials. We addressed this with the following strategies:

- **GPU Acceleration**: Ensured that training was done on a GPU (e.g., NVIDIA CUDA), drastically cutting training times.
- **Batch Size Tuning**: Larger batch sizes (e.g., 64 or 128) often speed up training by processing more examples in parallel, but can also influence optimization trajectories. We found that a batch size of 64 provided a good balance between speed and stability.

5.4 Challenge: Balancing Model Complexity

Deeper networks can capture more complex representations but can also overfit or become harder to train. We found that a moderately sized CNN with three blocks of convolutional layers, followed by a dense layer, worked well as a baseline. Testing more layers or filter sizes was a part of the optional architecture tuning, but we often found diminishing returns without extensive hyperparameter searches.

5.5 Solutions and Implementation Highlights

- **Incremental Prototyping**: We started with a simple CNN and gradually introduced layers, hyperparameters, and augmentation to see how each addition impacted performance.
- **Systematic Logging**: We kept track of each run’s settings (e.g., dropout rate, L2 penalty) in a structured format. This made it easier to replicate and compare experiments.
- **AutoML Tools**: Using Keras Tuner allowed for a more rigorous approach to hyperparameter search than manual tuning alone.

---

6. Limitations

While the pipeline achieves strong results, it’s essential to acknowledge some limitations:

1. **Dataset Constraints**: CIFAR-10, although standardized, is a relatively small dataset in terms of image size (32×32). This may limit the network's ability to learn finer details, and results do not necessarily transfer directly to high-resolution image tasks.
2. **Computational Budget**: Our hyperparameter search was relatively small in scope. Exploring a larger hyperparameter space or employing advanced search algorithms (e.g., Bayesian optimization, Hyperband) could yield better results, but at a higher computational cost.
3. **Architecture Choices**: We tested only a small subset of possible CNN architectures. More advanced architectures, like ResNet or DenseNet, could surpass our results but would require additional complexity in coding and experimentation.
4. **Limited Transfer Learning Experiments**: While we demonstrated an optional code snippet for MobileNetV2, we did not perform a comprehensive analysis of different pre-trained models (e.g., ResNet-50, Inception).

These limitations hint at potential directions for future work: expanding the data, employing more sophisticated search algorithms, and experimenting with deeper or more advanced architectures.

---

7. Conclusion

In this project, we presented an end-to-end deep learning pipeline for image classification, focusing on the CIFAR-10 dataset. We walked through data preprocessing, data augmentation, CNN architecture design, hyperparameter tuning, and comprehensive evaluation metrics. The integration of regularization strategies—dropout, L2 penalty, and early stopping—proved critical in mitigating overfitting. Furthermore, data augmentation helped the model generalize better, while hyperparameter tuning via Keras Tuner provided a more systematic path to improving performance.

Our final model achieved a test accuracy around 80–82%, which is noteworthy given the simplicity of the architecture and the moderate training time. The confusion matrix and classification report revealed that while the model performed well overall, it struggled with classes that share visual similarity, such as different types of animals or certain types of vehicles. Nevertheless, the pipeline offered here is generalizable and can be readily adapted to other datasets or tasks.

In closing, this project underscores the synergy between careful data augmentation, robust architectures, and methodical hyperparameter exploration. Future work might involve scaling the architecture, investigating additional regularization methods (such as batch normalization), or leveraging advanced transfer learning approaches with deeper pre-trained networks. By sharing this pipeline and methodology, we hope to contribute a reusable and transparent framework for practitioners and researchers eager to tackle real-world image classification challenges.

References

1. **Krizhevsky, A., Sutskever, I., & Hinton, G. E.** (2012). *ImageNet Classification with Deep Convolutional Neural Networks*. Communications of the ACM, 60(6), 84–90.

2. **He, K., Zhang, X., Ren, S., & Sun, J.** (2016). *Deep Residual Learning for Image Recognition*. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition* (CVPR).
3. **Chollet, F.** (2017). *Xception: Deep Learning with Depthwise Separable Convolutions*. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition* (CVPR).
4. **Howard, A. G., et al.** (2017). *MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications*. arXiv:1704.04861 [cs.CV].
5. **Szegedy, C., Ioffe, S., Vanhoucke, V., & Alemi, A.** (2017). *Inception-v4, Inception-ResNet and the Impact of Residual Connections on Learning*. In *Proceedings of the AAAI Conference on Artificial Intelligence*.
6. **Goodfellow, I., Bengio, Y., & Courville, A.** (2016). *Deep Learning*. MIT Press.
7. **Abadi, M., et al.** (2016). *TensorFlow: A System for Large-Scale Machine Learning*. In *OSDI*.
8. **Keras Tuner Documentation**. Retrieved from: https://keras.io/keras_tuner/